

OPERATING SYSTEM WITH LINUX (MCA-163)



Practical File

Submitted To:

Dr.Sunil Pratap Singh
(Associate Professor)

Submitted By:

Piyush Goel (09311604425)
MCA 1stSem,Sec B

INDEX

S. No.	ProblemDescription	Date	Sign.
P1	Install Virtual Box and then configure Linux(Ubantu) in VirtualBox.		
P2	Run and note the PID of your shell. Log out and login again and run ps again. What do you observe?		
P3	Enter the following commands, and note your observations: whoandtty (ii) tputclear (iii) id (iv) psandecho\$.		
P4	Run the following commands and then invoke ls. What do you conclude? echo > README [Enter] echo > readme [Enter]		
P5	Create a directory and change to that directory next create another directory in the new directory and then change to that directory too. Now run \$ cd without any arguments followed by pwd. What do you conclude?		
P6	Create a file mca containing the words "Hello MCA Class!". Now create a directory bvica, and then run mv mca bvica. What do you observe when you run both ls and ls bar?		
P7	Run \$ who am i and then interpret the output.		
P8	Find out whether the following commands are internal or external: echo, date, pwd, and ls.		
P9	Display the current date in the form dd/mm/yyyy.		
P10	Both of the following commands try to open the file mca but the error messages are a little different. What could be the reason? \$cat mca cat:mca:No such file or directory\$cat<mca bash: mca: No such file or directory.		
P11	Run the following commands and discuss their output? (a)\$uname (b)\$passwd (c)\$echo\$SHELL (d)\$man man (e) \$ which echo (f) \$ type echo (g) \$ whereis ls (h) \$ cd (i) \$ cd \$HOME (j) \$ cd ~		
P12	Use the find command to (i) mark directories and executables separately and (ii) also display hidden files.		
P13	Find out the result of following: \$ cat mca		
P14	Run the following and determine which commands will work? Explain with reasons. (a)\$ mkdir a/b/ (b) \$ mkdir a a/b (c) \$ rmdir a/b/c (d) \$ rmdir a a/b (e) \$ mkdir bin/mca		
P15	How does the command mv mca1,mca2 behave where both mca1 and mca2 are directories, when (i) mca2 exists (ii) mca2 doesn't exist?		

P16	Assuming that you are positioned in the directory/home/bvicam, what are these commands presumed to do, and explain whether they will		
-----	--	--	--

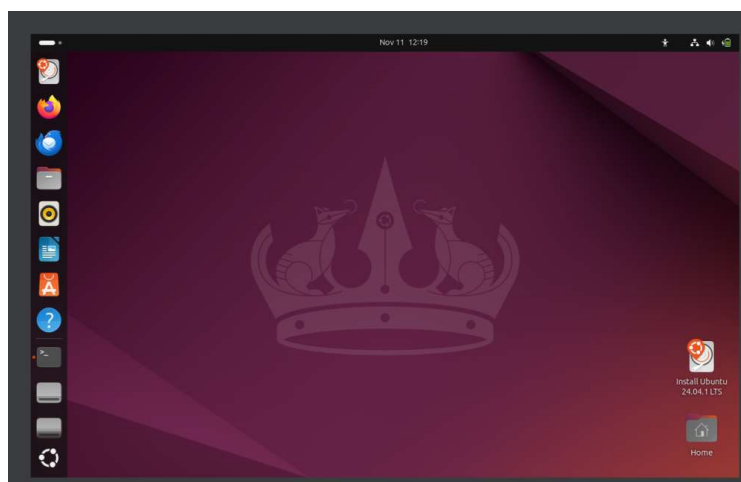
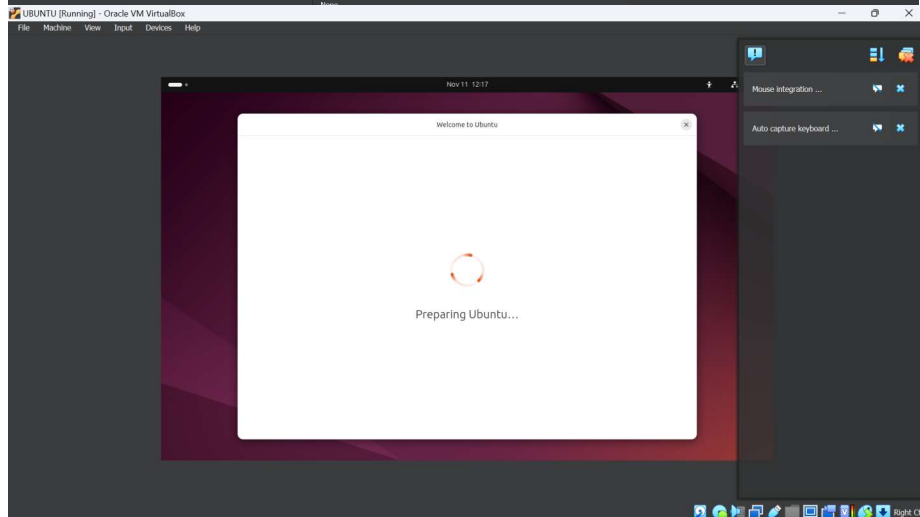
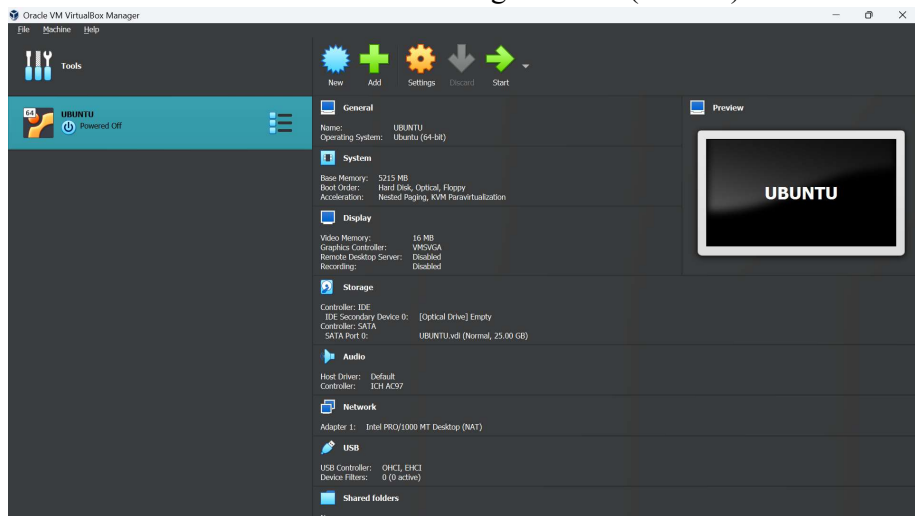
	work at all: (CO2) (a) \$ cd ../../ (b) \$ mkdir ../bin (c) \$ rmdir .. (d) \$ ls..		
--	---	--	--

P17	Apply Peterson algorithm for solving the critical section problem with C/Java multi-threaded programming. Assume appropriate code snippet for critical section.		
P18	Apply Bakery algorithm for synchronization of processes/threads in a C/Java program. Assume appropriate code snippet for critical section.		
P19	Write C/Java program to simulate and solve the Producer- Consumer problem.		
P20	Implement Semaphore(s) in a C/Java-multithreaded program to simulate the working and solution of Reader-Writer problem. Assume multiple reader and writers.		
P21	Create a zombie process and an orphan process in C program with appropriate system calls.		
P22	Write a C program which creates a new process and allows both, child and parent, to report their identification numbers (ids). The parent process should wait for the termination of The child process.		
P23	Write two „C“ programs (A.c and B.c) where one program (A.c) creates a child process and then that child process executes the code of other program(B.c). The logic of program „B.c“ is to generate all the prime numbers within the specified limit.		
P24	Write an appropriate C program which implements the concept of dynamic memory allocation (use of malloc(), calloc(), realloc(), and free() system call).		
P25	Create a text file, named as „courses.txt“ that contains the following four lines: Java Programming Operating System Structure Write a C program that forks three other processes. After forking, the parent process goes into wait state and waits for the children to finish their execution each child process reads a line from the course txt file (Child 1 Reads Line 1, Child 2 Reads Line 2, and Child 3 Reads Line 3) and each prints the respective line the lines can be printed in any order.		

P26	Write a C program (using appropriate system calls of Linux) that generate integers and stores the min a text file named a All.txt then retrieve the stored integers from this file and copy to "Odd.txt" and Even.txt" based upon the type of number, i.e if the retrieved integer is odd number then store in "Odd.txt" file or if the retrieved integer is even then store in "Even.txt" file. Finally, display the contents of all three files on the screen.		
P27	Write a program in "C" which accepts the file or directory name and permission (access rights) from the user and then changes the access rights accordingly. Use appropriate system call(s) of Linux.		
P28	Write a C program(using appropriate system calls of Linux) which generates and stores the characters from "a" to "z". Then display the stored characters in alternative manner like: a, c, e, g,..., etc.		
P29	Write a C program(using appropriate system calls of Linux) which receives roll number and names of n students, from the user one-by-one and then stores them in a text file, named as Student.txt after inserting all and roll number and names, display the contents of file. Also, display the access rights of the file Student.txt.		
P30	Demonstrate the use of following system calls by writing an appropriate "C" program. (a)lseek()(b)chmod()(c)umask()(d)access()(e)utime()		

Operating System with Linux Practical File (MCA-163)

P1. Install Virtual Box and then configure Linux(Ubuntu) in VirtualBox



P2.Run and note the PID of your shell.Log out and login again and run ps again. What do you observe?

Ps

```
piyush123@Ubuntu1:~$ ps
  PID TTY          TIME CMD
 3817 pts/0        00:00:00 bash
 3870 pts/0        00:00:00 ps
piyush123@Ubuntu1:~$
```

P3.Enter the following commands, and note your observations:
whoandtty (ii) tputclear (iii) id (iv) psandecho\$\$.

Ans)

Who (Shows a list of users currently logged into the system.)

tty (Displays the terminal (teletype) device filename that is connected to the user's session.)

```
piyush123@Ubuntu1:~$ who
piyush123  seat0          2025-11-26 16:21 (login screen)
piyush123  tty2           2025-11-26 16:21 (tty2)
piyush123@Ubuntu1:~$ tty
/dev/pts/0
piyush123@Ubuntu1:~$
```

tput clear (Clears the terminal screen)

```
piyush123@Ubuntu1:~$ tput clear
```

Operating System with Linux Practical File (MCA-163)

```
piyush123@Ubuntu1:~$
```

id (Prints user and group information for the current user, including user ID (UID), group ID (GID), and other groups.)

```
piyush123@Ubuntu1:~$ id
uid=1000(piush123) gid=1000(piush123) groups=1000(piush123),27(sudo)
piyush123@Ubuntu1:~$
```

ps (Lists active processes associated with the current terminal session.)

```
piyush123@Ubuntu1:~$ ps
  PID TTY          TIME CMD
 3817 pts/0        00:00:00 bash
 3885 pts/0        00:00:00 ps
piyush123@Ubuntu1:~$
```

echo \$\$ (Displays the process ID (PID) of the current shell.)

```
piyush123@Ubuntu1:~$ echo $$
3817
piyush123@Ubuntu1:~$
```

P4.Run the following commands and then invoke ls. What do you conclude?

echo > README [Enter] echo > readme [Enter]

echo > README

echo > readme

ls

```
piyush123@Ubuntu1:~$ echo > README
piyush123@Ubuntu1:~$ echo > readme
piyush123@Ubuntu1:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  readme  README  snap  Templates  Videos
piyush123@Ubuntu1:~$
```

P5.Create a directory and change to that directory next create another directory in the new directory and then change to that directory too. Now run \$ cd without any arguments followed by pwd. What do you conclude?

mkdir mydir

cd mydir

mkdir innerdir

cd innerdir

pwd

cd

cd (with no arguments): Changes the directory to the user's home directory.

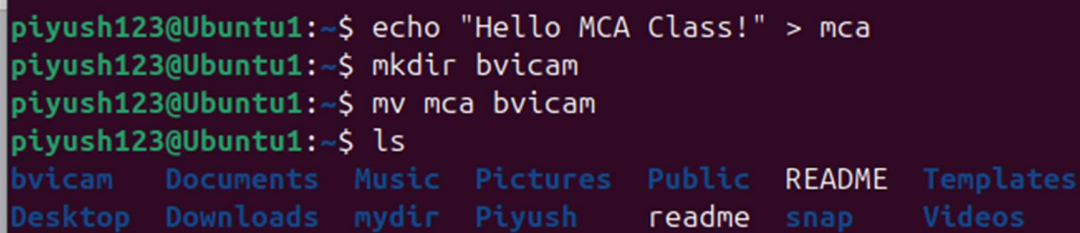
pwd: Displays the current working directory's full path.

```
piyush123@Ubuntu1:~$ mkdir Piyush
piyush123@Ubuntu1:~$ cd Piyush
piyush123@Ubuntu1:~/Piyush$ mkdir innerdir
piyush123@Ubuntu1:~/Piyush$ cd innerdir
piyush123@Ubuntu1:~/Piyush/innerdir$ pwd
/home/piyush123/Piyush/innerdir
piyush123@Ubuntu1:~/Piyush/innerdir$ cd
piyush123@Ubuntu1:~$
```

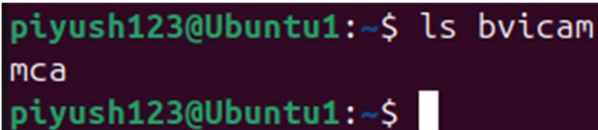

P6. Create a file mca containing the words "Hello MCA Class!". Now create a directory bvicam, and then run mv mca bvicam. What do you observe when you run both ls and ls bar?

```
echo "Hello MCA Class!" > mca
mkdir bvicam
mv mca bvicam
ls
ls bvicam
```

The mv command in Linux is used to move or rename files and directories.



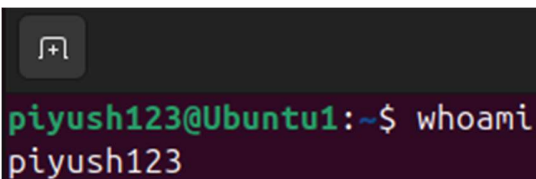
```
piyush123@Ubuntu1:~$ echo "Hello MCA Class!" > mca
piyush123@Ubuntu1:~$ mkdir bvicam
piyush123@Ubuntu1:~$ mv mca bvicam
piyush123@Ubuntu1:~$ ls
bvicam  Documents  Music  Pictures  Public  README  Templates
Desktop Downloads mydir  Piyush  readme  snap    Videos
```



```
piyush123@Ubuntu1:~$ ls bvicam
mca
piyush123@Ubuntu1:~$
```

P7. Run \$ who am i and then interpret the output.

```
whoami
```



```
piyush123@Ubuntu1:~$ whoami
piyush123
```

P8. Find out whether the following commands are internal or external: echo, date, pwd, and ls.

```
type echo
type date
type pwd
type ls
```

```
piyush123@Ubuntu1:~$ type echo
echo is a shell builtin
piyush123@Ubuntu1:~$ type date
date is /usr/bin/date
piyush123@Ubuntu1:~$ type pwd
pwd is a shell builtin
piyush123@Ubuntu1:~$ type ls
ls is aliased to `ls --color=auto'
piyush123@Ubuntu1:~$
```

P9. Display the current date in the form dd/mm/yyyy.

date +"%d/%m/%Y"

```
piyush123@Ubuntu1:~$ date
Wed Nov 26 05:05:08 PM UTC 2025
piyush123@Ubuntu1:~$ date +"%d/%m/%Y"
26/11/2025
```

P10. Both of the following commands try to open the file mca but the error messages are a little different. What could be the reason?

\$cat mca cat:mca:No such file or directory\$cat<mca bash: mca: No such file or directory.

cat mca

cat < mca

```
piyush123@Ubuntu1:~$ cat mca
cat: mca: No such file or directory
piyush123@Ubuntu1:~$ cat < mca
bash: mca: No such file or directory
piyush123@Ubuntu1:~$
```

the difference is:

- **cat mca:** Tries to open and read the file `mca` directly; it shows "No such file or directory" if `mca` doesn't exist.
- **cat < mca:** Redirects input from `mca`, but since the shell handles the redirection, it outputs "bash: mca: No such file or directory" if `mca` doesn't exist.

The main difference is in where the error message originates—`cat` vs. the shell.

P11. Run the following commands and discuss their output? (a) `$uname`

(b) `$passwd` (c) `$echo$SHELL` (d) `$man man`

(e) `$ which echo` (f) `$ type echo` (g) `$ whereis ls` (h) `$ cd`

(i) `$ cd $HOME` (j) `$ cd ~`

`uname`

`passwd`

`echo $SHELL`

`man man`

`which echo`

`type echo`

`whereis ls`

`cd`

`cd $HOME`

`cd ~`

brief explanation of each command used in P11:

1. **uname:** Shows system information (e.g., operating system name). Use `uname -a` for full details.
2. **passwd:** Changes the user's password. Prompts for the old and new password.
3. **echo \$SHELL:** Displays the path of the current shell in use (e.g., `/bin/bash`).
4. **man man:** Opens the manual page for the `man` command itself, explaining how to use the manual.
5. **which echo:** Shows the path to the `echo` command executable, indicating whether it's built-in or an external command.
6. **type echo:** Displays whether `echo` is a shell built-in or an external command, with details.
7. **whereis ls:** Locates the binary, source, and manual files for `ls`.
8. **cd:** Changes to the user's home directory when no path is specified.
9. **cd \$HOME:** Changes the directory to the user's home directory explicitly.
10. **cd ~:** Also changes to the user's home directory (`~` is a shortcut for the home directory).

Operating System with Linux Practical File (MCA-163)

```
piyush123@Ubuntu1: ~/Desktop
piyush123@Ubuntu1:~$ uname
Linux
piyush123@Ubuntu1:~$ passwd
Changing password for piyush123.
Current password:
New password:
BAD PASSWORD: The password is shorter than 8 characters
New password:
BAD PASSWORD: The password is the same as the old one
New password:
BAD PASSWORD: The password fails the dictionary check - it is too simplistic/systematic
passwd: Have exhausted maximum number of retries for service
passwd: password unchanged
piyush123@Ubuntu1:~$ echo $SHELL
/bin/bash
piyush123@Ubuntu1:~$ man man
man: can't resolve man7/groff_man.7
piyush123@Ubuntu1:~$ which echo
/usr/bin/echo
piyush123@Ubuntu1:~$ type echo
echo is a shell builtin
piyush123@Ubuntu1:~$ whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
piyush123@Ubuntu1:~$ cd
piyush123@Ubuntu1:~$ cd $HOME
piyush123@Ubuntu1:~$ cd Desktop
piyush123@Ubuntu1:~/Desktop$ cd $HOME
piyush123@Ubuntu1:~$ cd -
/home/piyush123/Desktop
piyush123@Ubuntu1:~/Desktop$
```

```
MAN(1)                                Manual pager utils                                MAN(1)

NAME
    man - an interface to the system reference manuals

SYNOPSIS
    man [man options] [section] page ...] ...
    man -k [apropos options] regexp ...
    man -K [man options] [section] term ...
    man -f [what is options] page ...
    man -l [man options] file ...
    man -w|-W [man options] page ...

DESCRIPTION
    man is the system's manual pager. Each page argument given to man is normally the name of a program, utility or function. The manual page associated with each of these arguments is then found and displayed. A section, if provided, will direct man to look only in that section of the manual. The default action is to search in all of the available sections following a pre-defined order (see DEFAULTS), and to show only the first page found, even if page exists in several sections.

    The table below shows the section numbers of the manual followed by the types of pages they contain.
```

P12.Frernels command to (i) mark directories and executables separately and (ii) also display hidden files.

`ls -F`

`ls -a`

`ls -af`

- **ls -F:** Marks directories (/) and executables (*).
- **ls -a:** Shows all files, including hidden ones.
- **ls -aF:** Combines both to show hidden files with type indicators.

```
piyush123@Ubuntu1: ~  
piyush123@Ubuntu1:~$ ls -F  
bvicam/  Documents/ Music/  Pictures/ Public/  README  Templates/  
Desktop/ Downloads/ mydir/ Piyush/  readme  snap/   Videos/  
piyush123@Ubuntu1:~$ ls -a  
..          .bashrc  .config  Downloads .local  Pictures Public  snap      Videos  
bvicam      Desktop  .gnupg   Music    Piyush  readme  .ssh  
.bash_logout .cache  Documents .lessht  mydir   .profile README  Templates  
piyush123@Ubuntu1:~$ ls -af  
Piyush .local  README .profile Videos  readme  Desktop  Templates .ssh  
..     bvicam  .bash_logout .gnupg  .lessht Pictures .        Music  
mydir  Documents .config  Public  snap    .cache  .bashrc  Downloads  
piyush123@Ubuntu1:~$
```

P13.Find out the result of following:

`$ cat mca`

```
piyush123@Ubuntu1:~$ cat mca mca mca  
cat: mca: No such file or directory  
cat: mca: No such file or directory  
cat: mca: No such file or directory  
piyush123@Ubuntu1:~$
```

P14. Run the following and determine which commands will work? Explain with reasons.

- (a) \$ mkdir a/b/ (b) \$ mkdir a a/b (c) \$ rmdir a/b/c (d) \$ rmdir a a/b
(e) \$ mkdir/bin/mca

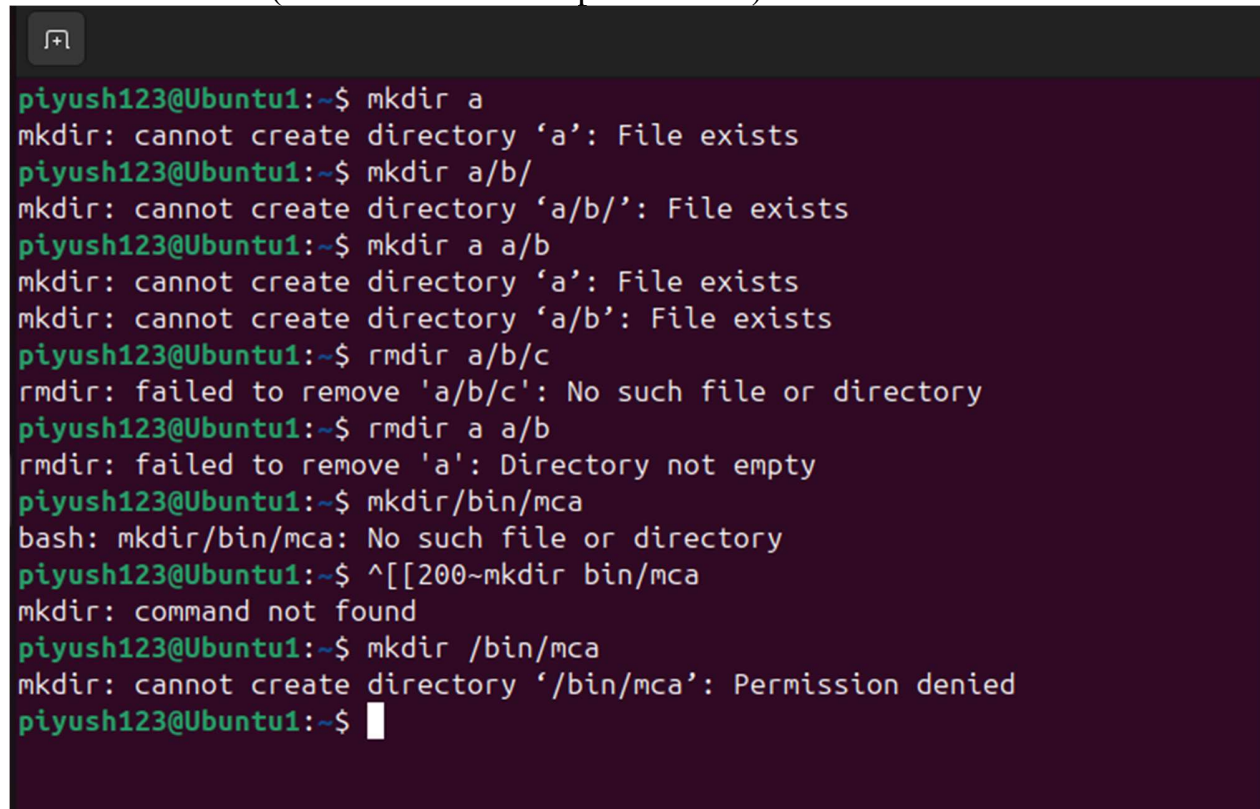
mkdir -p a/b

mkdir a a/b

rmdir a/b/c

rmdir a a/b

mkdir /bin/mca # (This will fail due to permissions)

A terminal window with a dark purple background and light green text. It shows the execution of several commands and their outputs. The prompt is 'piyush123@Ubuntu1:~\$'. The commands and their outputs are: 'mkdir a' results in 'mkdir: cannot create directory 'a': File exists'; 'mkdir a/b/' results in 'mkdir: cannot create directory 'a/b/': File exists'; 'mkdir a a/b' results in 'mkdir: cannot create directory 'a': File exists' and 'mkdir: cannot create directory 'a/b': File exists'; 'rmdir a/b/c' results in 'rmdir: failed to remove 'a/b/c': No such file or directory'; 'rmdir a a/b' results in 'rmdir: failed to remove 'a': Directory not empty'; 'mkdir/bin/mca' results in 'bash: mkdir/bin/mca: No such file or directory'; '^[[200~mkdir bin/mca' results in 'mkdir: command not found'; 'mkdir /bin/mca' results in 'mkdir: cannot create directory '/bin/mca': Permission denied'. The prompt is followed by a cursor character '█'.

(a) \$ mkdir a/b/

Outcome: This command will fail if the directory a does not already exist.

Reason: The mkdir command will only create the specified directory structure if the parent

directories exist. To create a/b/ in one command without errors, you should use

mkdir -p a/b/

to ensure that any necessary parent directories are created.

(b) \$ mkdir a a/b

Outcome: This command will work.

Reason: The command first creates the directory a and then a/b inside a. If a already exists,

the command will still proceed without error and create a/b inside it.

(c) `$ rmdir a/b/c`

Outcome: This command will fail if c does not exist or if c is not empty.

Reason: The rmdir command only removes empty directories. If a/b/c is not an existing

empty directory, this command will not work.

(d) `$ rmdir a a/b`

Outcome: This command will fail if either a or a/b is not empty.

Reason: The rmdir command can remove multiple directories in one command but only if

they are all empty. If a or a/b contains any files or subdirectories, the command will not work.

(e) `$ mkdir /bin/mca`

Outcome: This command will fail for most users.

Reason: The /bin directory is part of the system's essential directories, and normal users do

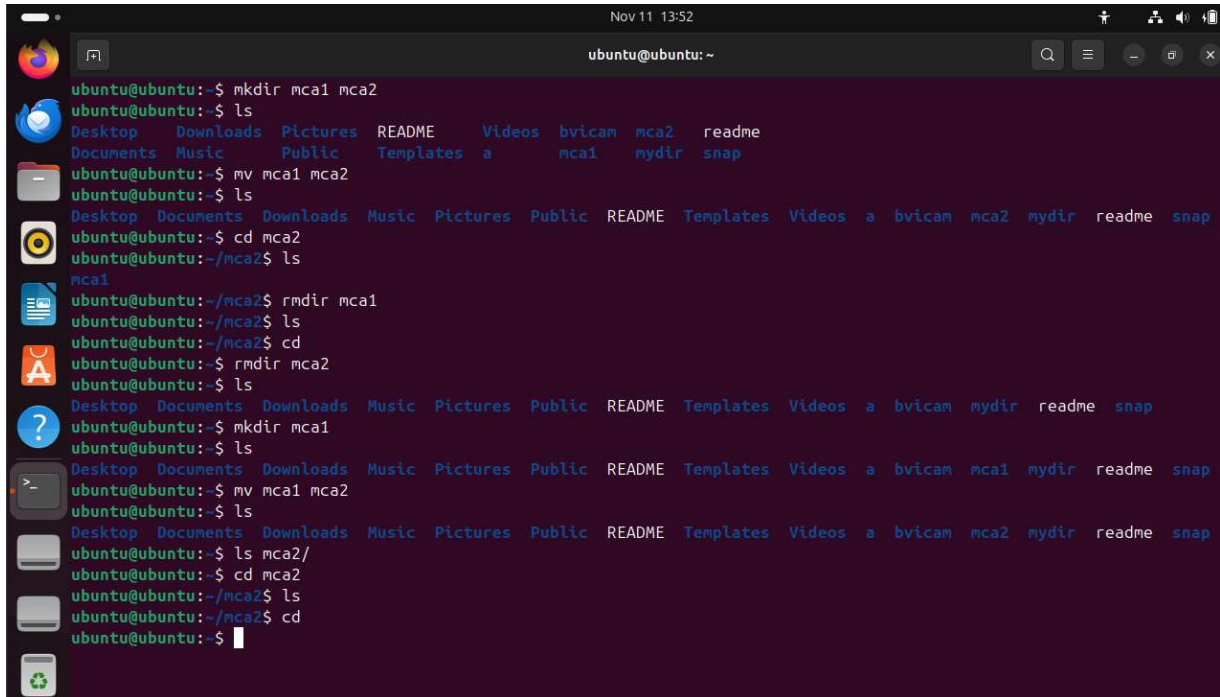
not have write permissions to create directories within it. To run this command successfully,

superuser privileges (using sudo) are required: `sudo mkdir /bin/mca`. However, it is not

recommended to create user directories within /bin as it is reserved for essential system

binaries.

P15.How does the command `mv mca1,mca2` behave where both `mca1` and `mca2` are directories, when (i) `mca2` exists (ii) `mca2` doesn't exist?
`mv mca1 mca2`

A terminal window titled 'ubuntu@ubuntu: ~' showing a series of commands and their outputs. The user creates 'mca1' and 'mca2' directories, then moves 'mca1' into 'mca2'. The terminal output shows the directory structure before and after the move, confirming that 'mca1' is now a subdirectory of 'mca2'.

```
ubuntu@ubuntu:~$ mkdir mca1 mca2
ubuntu@ubuntu:~$ ls
Desktop  Downloads  Pictures  README    Videos  bvicam  mca2  readme
Documents Music      Public   Templates a        mca1   mydir snap
ubuntu@ubuntu:~$ mv mca1 mca2
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  README  Templates  Videos  a  bvicam  mca2  mydir  readme  snap
ubuntu@ubuntu:~$ cd mca2
ubuntu@ubuntu:~/mca2$ ls
mca1
ubuntu@ubuntu:~/mca2$ rmdir mca1
ubuntu@ubuntu:~/mca2$ ls
ubuntu@ubuntu:~/mca2$ cd
ubuntu@ubuntu:~$ rmdir mca2
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  README  Templates  Videos  a  bvicam  mydir  readme  snap
ubuntu@ubuntu:~$ mkdir mca1
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  README  Templates  Videos  a  bvicam  mca1  mydir  readme  snap
ubuntu@ubuntu:~$ mv mca1 mca2
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  README  Templates  Videos  a  bvicam  mca2  mydir  readme  snap
ubuntu@ubuntu:~$ ls mca2/
ubuntu@ubuntu:~$ cd mca2
ubuntu@ubuntu:~/mca2$ ls
ubuntu@ubuntu:~/mca2$ cd
ubuntu@ubuntu:~$
```

(i) `mca2` exists

Outcome: The command `mv mca1 mca2` will move the directory `mca1` into `mca2` as a subdirectory.

Explanation: When `mca2` already exists and is a directory, the `mv` command will move `mca1`

(along with its contents) inside `mca2`, resulting in `mca2/mca1`.

(ii) `mca2` doesn't exist

Outcome: The command `mv mca1 mca2` will rename the directory `mca1` to `mca2`.

Explanation: If `mca2` does not exist, `mv` simply renames `mca1` to `mca2`. The original

directory `mca1` will no longer exist, and `mca2` will take its place with all the contents intact.

P16.

- `cd ../../`: Moves up two directories from the current directory.
- `mkdir ../bin`: Attempts to create a bin directory one level up from the current directory.
- `rmdir ..`: Tries to remove the parent directory, which will fail because you can't remove a directory while inside it.
- `ls ..`: Lists the contents of the parent directory.

```
piyush123@Ubuntu1:~/home
piyush123@Ubuntu1:~$ ls
a      Desktop  Downloads  mydir   Piyush  readme  snap      Videos
bvicam Documents  Music     Pictures Public  README  Templates
piyush123@Ubuntu1:~$ mkdir home
piyush123@Ubuntu1:~$ cd home
piyush123@Ubuntu1:~/home$ mkdir bvica
piyush123@Ubuntu1:~/home$ cd ../../
piyush123@Ubuntu1:/home$
```

```
ubuntu@ubuntu:~$ cd home/bvicam/
ubuntu@ubuntu:~/home/bvicam$ mkdir ../bin
ubuntu@ubuntu:~/home/bvicam$ cd .
ubuntu@ubuntu:~/home/bvicam$ cd..
cd..: command not found
ubuntu@ubuntu:~/home/bvicam$ cd ..
ubuntu@ubuntu:~/home$ ls
bin  bvica
ubuntu@ubuntu:~/home$
```

```
ubuntu@ubuntu:~/home$ cd bvica
ubuntu@ubuntu:~/home/bvicam$ rmdir ..
rmdir: failed to remove '..': Directory not empty
ubuntu@ubuntu:~/home/bvicam$ ls ..
bin  bvica
ubuntu@ubuntu:~/home/bvicam$
```

P17.Apply Peterson algorithm for solving the critical section problem with C/Java multi-threaded programming. Assume appropriate code snippet for critical section

```
#include <pthread.h>
#include <stdio.h>
#include <stdbool.h>

bool flag[2] = {false, false};
int turn = 0;
int shared_var = 0;

void *process0() {
    flag[0] = true;
    turn = 1;
    while (flag[1] && turn == 1);
    // Critical section
    shared_var++;
    printf("Process 0 in critical section: shared_var = %d\n", shared_var);
    flag[0] = false;
    return NULL;
}

void *process1() {
    flag[1] = true;
    turn = 0;
    while (flag[0] && turn == 0);
    // Critical section
    shared_var++;
    printf("Process 1 in critical section: shared_var = %d\n", shared_var);
    flag[1] = false;
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, process0, NULL);
    pthread_create(&t2, NULL, process1, NULL);
```

```
pthread_join(t1, NULL);
pthread_join(t2, NULL);
printf("Final value of shared_var = %d\n", shared_var);
return 0;
}
```

```
piyush123@Ubuntu1: ~/Desktop/OSFILE
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q17.c
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
Thread 0 is in critical section
Thread 1 is in critical section
Thread 0 is in critical section
Thread 1 is in critical section
Thread 0 is in critical section
Thread 1 is in critical section
Thread 0 is in critical section
Thread 1 is in critical section
Thread 0 is in critical section
Thread 1 is in critical section
piyush123@Ubuntu1:~/Desktop/OSFILE$
```

P18.Apply Bakery algorithm for synchronization of processes/threads in a C/Java program. Assume appropriate code snippet for critical section.

```
#include <pthread.h>
#include <stdio.h>
#include <stdbool.h>
#include <unistd.h>

#define N 2
bool choosing[N] = {false, false};
int ticket[N] = {0, 0};
int shared_var = 0;

int max_ticket() {
    int max = 0;
    for (int i = 0; i < N; i++) {
        if (ticket[i] > max) max = ticket[i];
    }
    return max;
}

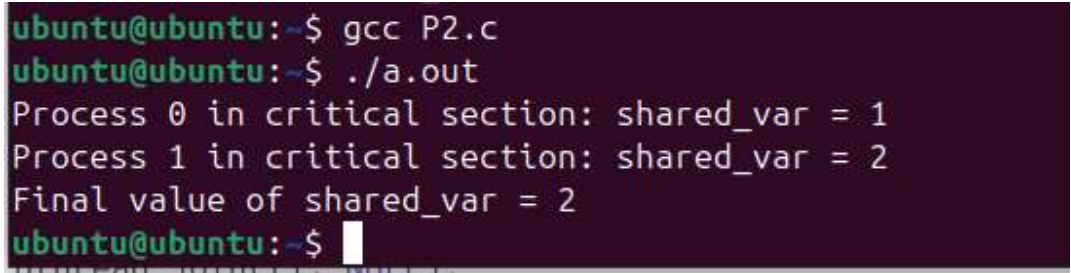
void lock(int i) {
    choosing[i] = true;
    ticket[i] = max_ticket() + 1;
    choosing[i] = false;
    for (int j = 0; j < N; j++) {
        while (choosing[j]);
        while (ticket[j] != 0 && (ticket[j] < ticket[i] || (ticket[j] == ticket[i] && j <
i)));
    }
}

void unlock(int i) {
    ticket[i] = 0;
}

void *process(void *arg) {
    int id = *((int *)arg);
    lock(id);
```

```
// Critical section
shared_var++;
printf("Process %d in critical section: shared_var = %d\n", id, shared_var);
unlock(id);
return NULL;
}

int main() {
    pthread_t t1, t2;
    int id0 = 0, id1 = 1;
    pthread_create(&t1, NULL, process, &id0);
    pthread_create(&t2, NULL, process, &id1);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Final value of shared_var = %d\n", shared_var);
    return 0;
}
```



```
ubuntu@ubuntu:~$ gcc P2.c
ubuntu@ubuntu:~$ ./a.out
Process 0 in critical section: shared_var = 1
Process 1 in critical section: shared_var = 2
Final value of shared_var = 2
ubuntu@ubuntu:~$
```

P19. Write C/Java program to simulate and solve the Producer- Consumer problem.

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;

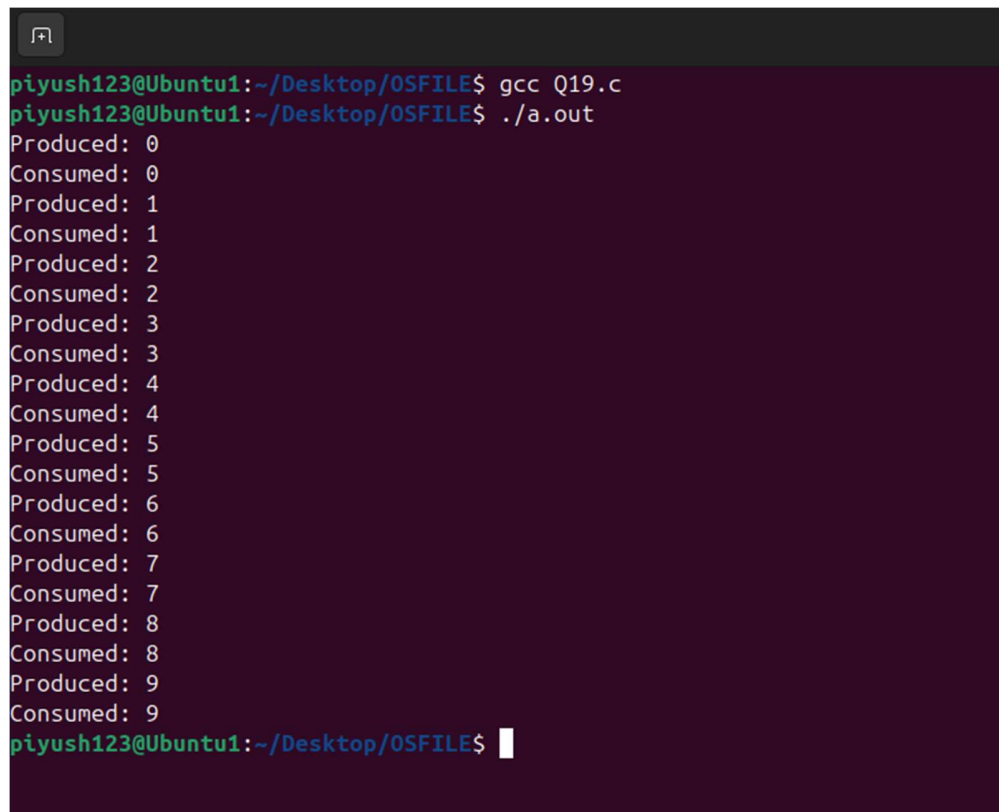
sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 0; i < 10; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 10; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(1);
    }
}
```

```
    return NULL;
}

int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

A terminal window with a dark purple background. The prompt is 'piyush123@Ubuntu1:~/Desktop/OSFILE\$'. The user enters 'gcc Q19.c' and then './a.out'. The program outputs a sequence of 'Produced' and 'Consumed' messages, showing that 10 items were produced and consumed in an interleaved fashion. The final prompt is 'piyush123@Ubuntu1:~/Desktop/OSFILE\$' with a cursor.

```
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q19.c
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
Produced: 0
Consumed: 0
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
piyush123@Ubuntu1:~/Desktop/OSFILE$
```

P20.Implement Semaphore(s)in a C/Java-multithreaded program to simulate the working and solution of Reader-Writer problem.Assume multiple reader sand writers.

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

sem_t mutex, db;
int reader_count = 0;

void *writer(void *arg) {
    sem_wait(&db);
    printf("Writer is writing\n");
    sem_post(&db);
    return NULL;
}

void *reader(void *arg) {
    sem_wait(&mutex);
    reader_count++;
    if (reader_count == 1) sem_wait(&db);
    sem_post(&mutex);

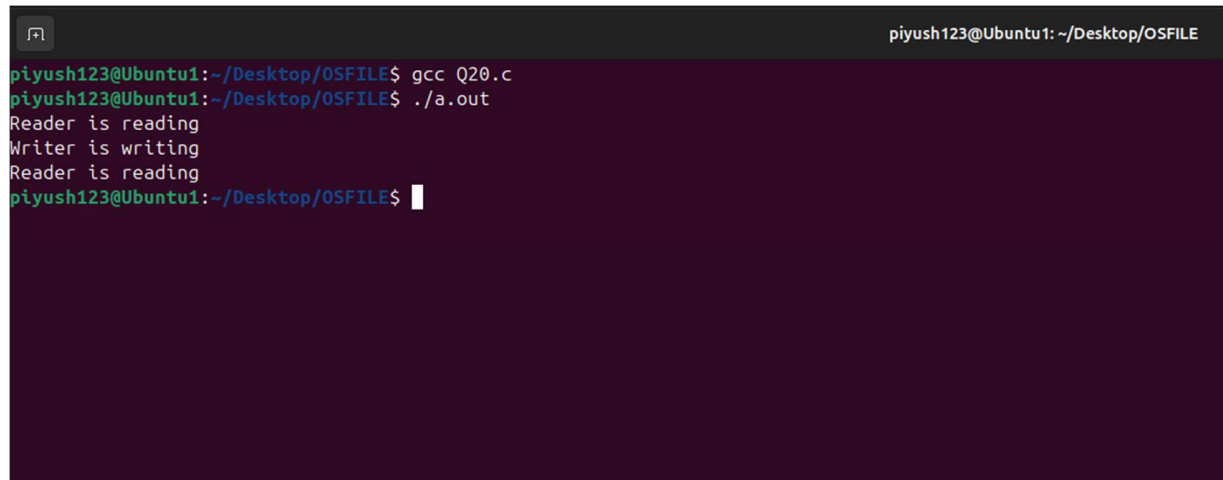
    printf("Reader is reading\n");

    sem_wait(&mutex);
    reader_count--;
    if (reader_count == 0) sem_post(&db);
    sem_post(&mutex);
    return NULL;
}

int main() {
    pthread_t r1, r2, w1;
    sem_init(&mutex, 0, 1);
    sem_init(&db, 0, 1);
    pthread_create(&r1, NULL, reader, NULL);
```



```
pthread_create(&r2, NULL, reader, NULL);  
pthread_create(&w1, NULL, writer, NULL);  
pthread_join(r1, NULL);  
pthread_join(r2, NULL);  
pthread_join(w1, NULL);  
sem_destroy(&mutex);  
sem_destroy(&db);  
return 0;  
}
```



```
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q20.c  
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out  
Reader is reading  
Writer is writing  
Reader is reading  
piyush123@Ubuntu1:~/Desktop/OSFILE$
```

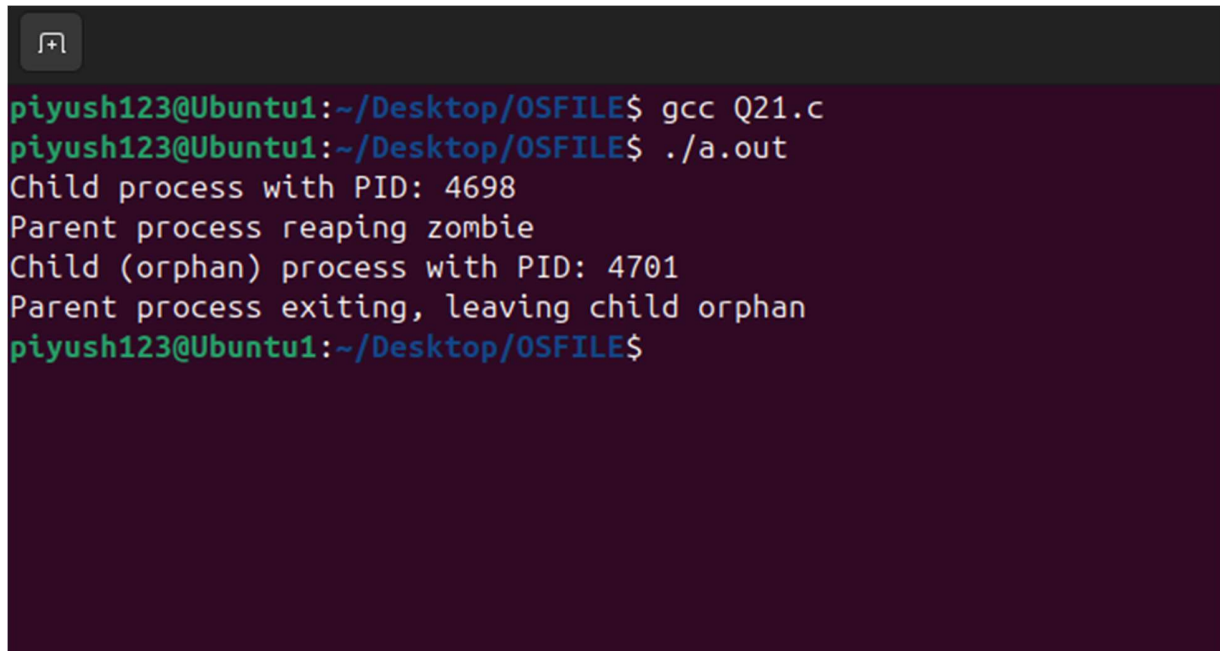
P21. Create a zombie process and an orphan processing C program with appropriate system calls.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void create_zombie() {
    if (fork() == 0) {
        printf("Child process with PID: %d\n", getpid());
        exit(0); // child exits, becoming a zombie until parent reaps it
    } else {
        sleep(5); // allow the child to become a zombie
        printf("Parent process reaping zombie\n");
    }
}

void create_orphan() {
    if (fork() == 0) {
        printf("Child (orphan) process with PID: %d\n", getpid());
        sleep(10); // orphaned when parent exits
    } else {
        printf("Parent process exiting, leaving child orphan\n");
        exit(0);
    }
}

int main() {
    create_zombie();
    create_orphan();
    return 0;
}
```

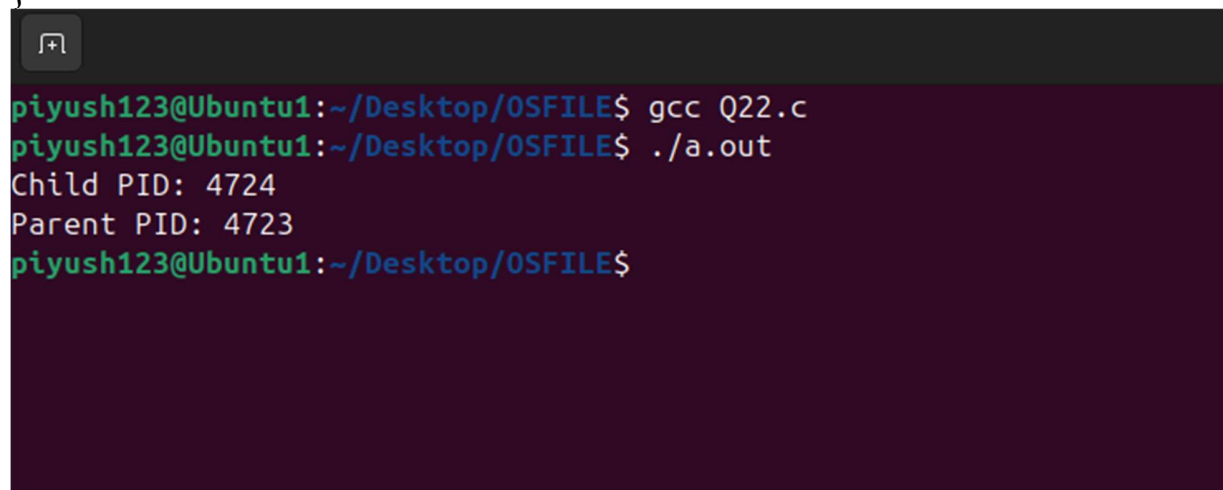


```
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q21.c
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
Child process with PID: 4698
Parent process reaping zombie
Child (orphan) process with PID: 4701
Parent process exiting, leaving child orphan
piyush123@Ubuntu1:~/Desktop/OSFILE$
```

P22. Write a C program which creates a new process and allows both, child and parent, to report their identification numbers (ids). The parent process should wait for the termination of the child process.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid > 0) {
        wait(NULL); // Parent waits for child
        printf("Parent PID: %d\n", getpid());
    } else if (pid == 0) {
        printf("Child PID: %d\n", getpid());
    }
    return 0;
}
```

A terminal window with a dark background and light green text. The prompt is 'piyush123@Ubuntu1:~/Desktop/OSFILE\$'. The user enters 'gcc Q22.c' and the prompt changes to 'piyush123@Ubuntu1:~/Desktop/OSFILE\$./a.out'. The program outputs 'Child PID: 4724' followed by 'Parent PID: 4723'. The prompt returns to 'piyush123@Ubuntu1:~/Desktop/OSFILE\$'.

```
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q22.c
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
Child PID: 4724
Parent PID: 4723
piyush123@Ubuntu1:~/Desktop/OSFILE$
```

P23. Write two „C“ programs (A.c and B.c) where one program (A.c) creates a child process and then that child process executes the code of other program(B.c). The logic of program „B.c“ is to generate all the prime numbers within the specified limit.

A.c

```
#include <stdio.h>
#include <unistd.h>

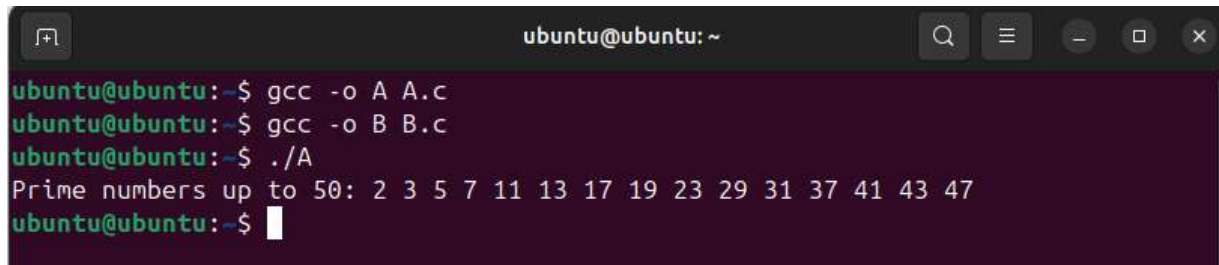
int main() {
    if (fork() == 0) {
        execl("./B", "./B", NULL); // Execute B program
    }
    return 0;
}
```

B.c

```
#include <stdio.h>
#include <stdlib.h>

void generate_primes(int limit) {
    for (int i = 2; i <= limit; i++) {
        int is_prime = 1;
        for (int j = 2; j * j <= i; j++) {
            if (i % j == 0) {
                is_prime = 0;
                break;
            }
        }
        if (is_prime) {
            printf("%d ", i);
        }
    }
    printf("\n");
}
```

```
int main() {  
    int limit = 50;  
    printf("Prime numbers up to %d: ", limit);  
    generate_primes(limit);  
    return 0;  
}
```



A terminal window titled 'ubuntu@ubuntu: ~' with standard window controls. The terminal shows the following commands and output:

```
ubuntu@ubuntu:~$ gcc -o A A.c  
ubuntu@ubuntu:~$ gcc -o B B.c  
ubuntu@ubuntu:~$ ./A  
Prime numbers up to 50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47  
ubuntu@ubuntu:~$
```

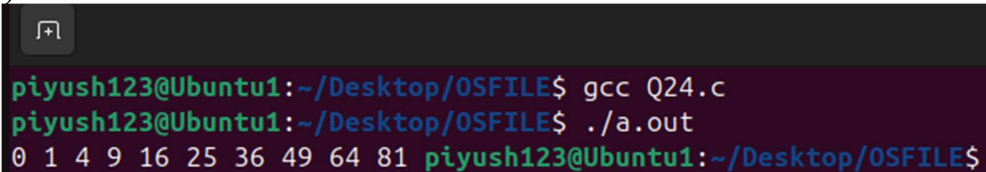
P24. Write an appropriate C program which implements the concept of dynamic memory allocation (use of malloc(), calloc(), realloc(), and free() system call).

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int*) malloc(5 * sizeof(int));
    for (int i = 0; i < 5; i++) arr[i] = i * i;

    arr = realloc(arr, 10 * sizeof(int));
    for (int i = 5; i < 10; i++) arr[i] = i * i;

    for (int i = 0; i < 10; i++) printf("%d ", arr[i]);
    free(arr);
    return 0;
}
```



```
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q24.c
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
0 1 4 9 16 25 36 49 64 81 piyush123@Ubuntu1:~/Desktop/OSFILE$
```

P25. Create a text file, named as „courses.txt“ that contains the following four lines:
Java Programming Operating System Structure Write a C program that forks three other processes. After forking, the parent process goes into wait state and waits for the children to finish their execution each child process reads a line from the course txt file (Child 1 Reads Line 1, Child 2 Reads Line 2, and Child 3 Reads Line 3) and each prints the respective line the lines can be printed in any order.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    FILE *file = fopen("courses.txt", "r");
    if (!file) {
        perror("Error opening file");
        return 1;
    }

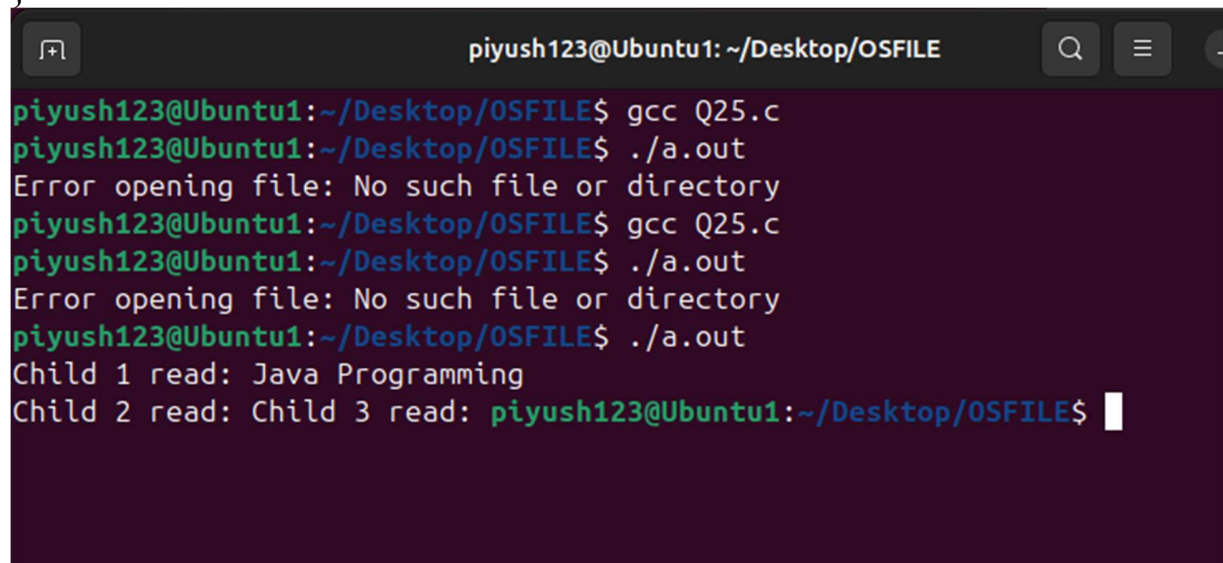
    char line[100];

    // Fork 3 child processes
    for (int i = 0; i < 3; i++) {
        if (fork() == 0) { // In child process
            // Each child reads the corresponding line from the file
            for (int j = 0; j <= i; j++) {
                fgets(line, sizeof(line), file); // Read one line at a time
            }
            printf("Child %d read: %s", i + 1, line); // Print the line read by the child
            fclose(file); // Close file in child process
            exit(0); // Exit child process
        }
    }

    // Parent waits for all children to finish
    fclose(file); // Close the file in the parent process
    for (int i = 0; i < 3; i++) {
        wait(NULL); // Wait for any child to finish
    }
}
```



```
    return 0;  
}
```



A terminal window titled "piyush123@Ubuntu1: ~/Desktop/OSFILE" with a search icon and a menu icon in the top right corner. The terminal shows the following commands and output:

```
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q25.c  
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out  
Error opening file: No such file or directory  
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q25.c  
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out  
Error opening file: No such file or directory  
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out  
Child 1 read: Java Programming  
Child 2 read: Child 3 read: piyush123@Ubuntu1:~/Desktop/OSFILE$
```

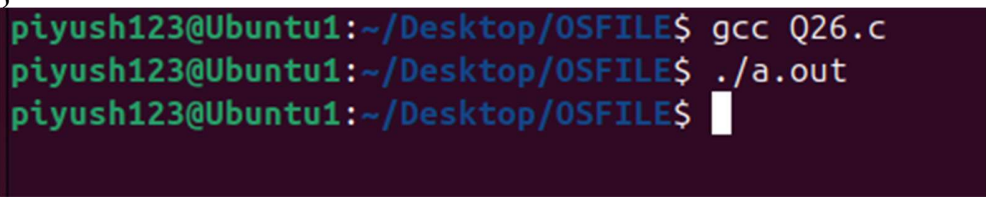
P26. Write a C program (using appropriate system calls of Linux) that generate and stores the min a text file named a All.txt then retrieve the stored integers from this file and copy to "Odd.txt" and Even.txt" based upon the type of number, i.e if the retrieved integer if odd number then store in „Odd.txt" file or if the retrieved integer is even then store in „Even.txt" file. Finally, display the contents of all three files on the screen

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *all = fopen("All.txt", "w");
    for (int i = 1; i <= 10; i++) fprintf(all, "%d\n", i);
    fclose(all);

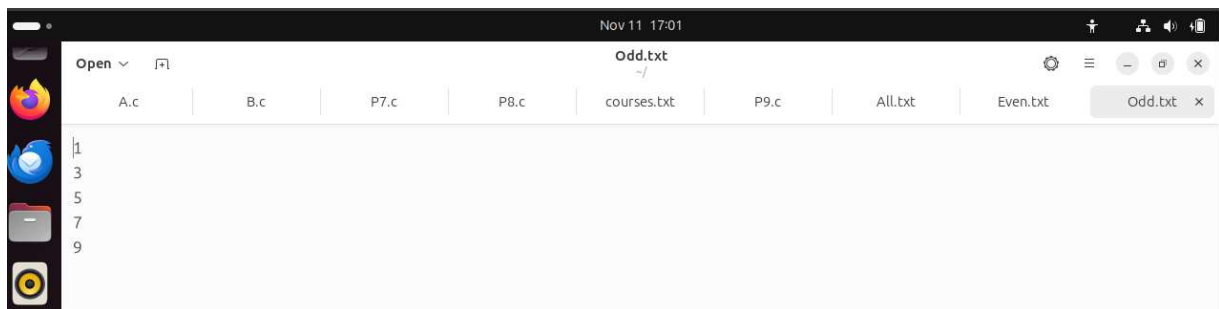
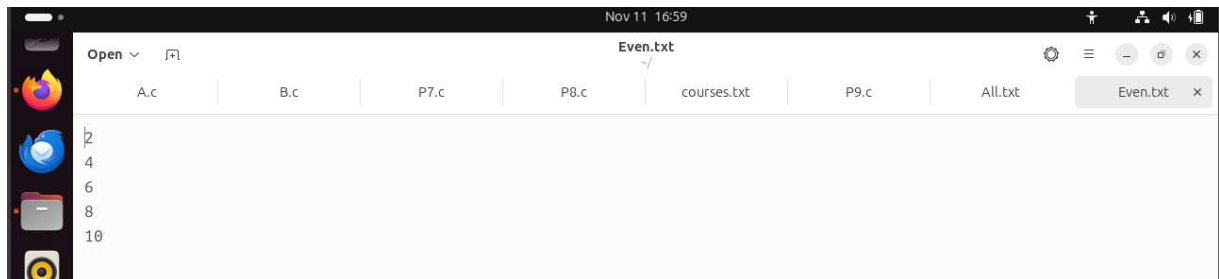
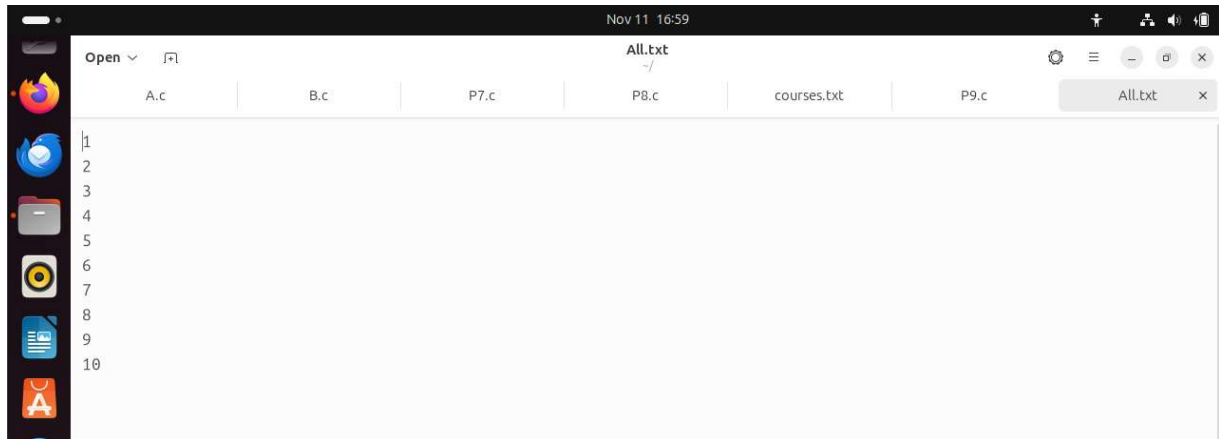
    FILE *odd = fopen("Odd.txt", "w"), *even = fopen("Even.txt", "w");
    all = fopen("All.txt", "r");

    int num;
    while (fscanf(all, "%d", &num) != EOF) {
        if (num % 2 == 0) fprintf(even, "%d\n", num);
        else fprintf(odd, "%d\n", num);
    }
    fclose(all);
    fclose(odd);
    fclose(even);
    return 0;
}
```



```
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q26.c
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
piyush123@Ubuntu1:~/Desktop/OSFILE$
```

Operating System with Linux Practical File (MCA-163)



P27. Write a program in „C“ which accepts the file or directory name and permission (access rights) from the user and then changes the access rights accordingly. Use appropriate system call(s) of Linux.

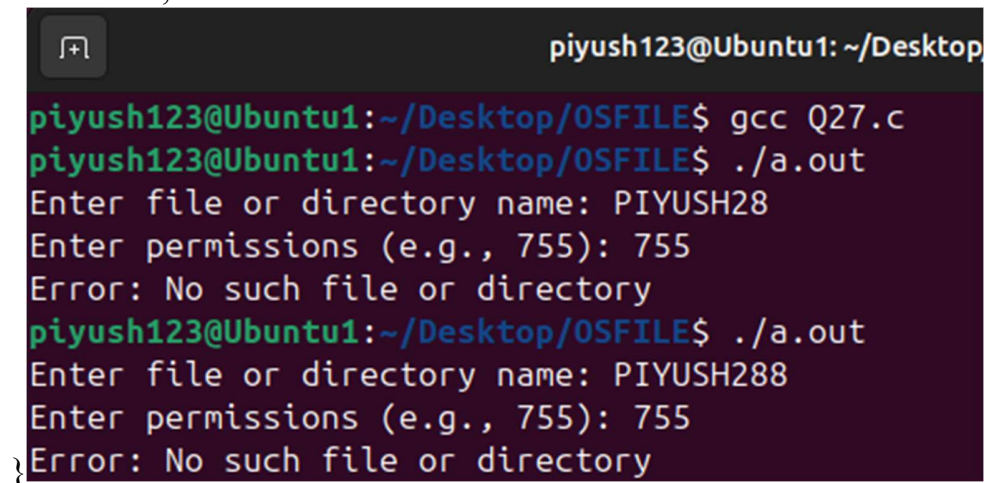
```
#include <stdio.h>
#include <sys/stat.h>

int main() {
    char filename[256];
    int permissions;

    // Get file/directory name and new permissions
    printf("Enter file or directory name: ");
    scanf("%s", filename);
    printf("Enter permissions (e.g., 755): ");
    scanf("%o", &permissions);

    // Change permissions
    if (chmod(filename, permissions) == 0) {
        printf("Permissions changed successfully.\n");
    } else {
        perror("Error");
    }

    return 0;
}
```



```
piyush123@Ubuntu1: ~/Desktop
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q27.c
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
Enter file or directory name: PIYUSH28
Enter permissions (e.g., 755): 755
Error: No such file or directory
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
Enter file or directory name: PIYUSH288
Enter permissions (e.g., 755): 755
Error: No such file or directory
```

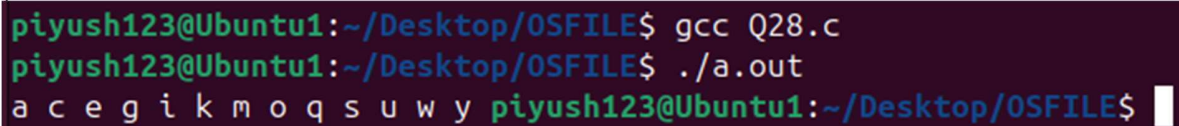
P28. Write a C program(using appropriate system calls of Linux) which generates and stores the characters from a“ to „z“.

Then display the stored characters in alternative manner like: a, c, e, g,..., etc.

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    FILE *file = fopen("alphabet.txt", "w");
    for (char c = 'a'; c <= 'z'; c++) fputc(c, file);
    fclose(file);

    file = fopen("alphabet.txt", "r");
    for (int i = 0; i <= 25; i += 2) {
        fseek(file, i, SEEK_SET);
        printf("%c ", fgetc(file));
    }
    fclose(file);
    return 0;
}
```



```
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q28.c
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
a c e g i k m o q s u w y piyush123@Ubuntu1:~/Desktop/OSFILE$
```

P29. Write a C program (using appropriate system calls of Linux) which receives roll number and names of n students, from the user one-by-one and then stores them in a text file, named as Student.txt after inserting all and roll number and names, display the contents of file. Also, display the access rights of the file Student.txt.

```
#include <stdio.h>
#include <sys/stat.h>

int main() {
    FILE *file = fopen("Student.txt", "w");
    int n;
    printf("Enter number of students: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        int roll;
        char name[20];
        printf("Enter roll and name: ");
        scanf("%d %s", &roll, name);
        fprintf(file, "%d %s\n", roll, name);
    }
    fclose(file);

    struct stat st;
    stat("Student.txt", &st);
    printf("File permissions: %o\n", st.st_mode & 0777);
    return 0;
}
```

The screenshot shows a terminal window titled 'piyush123@Ubuntu1: ~/Desktop/OSFILE'. The user runs the following commands:

```
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q29.c
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
Enter number of students: 2
Enter roll and name: 1
Piyush
Enter roll and name: 2
Aadish
File permissions: 664
piyush123@Ubuntu1:~/Desktop/OSFILE$
```

Below the terminal, a file editor window titled 'Student.txt' is open, showing the contents of the file:

```
1 Piyush
2 Aadish
```

P30. Demonstrate the use of following system calls by writing an appropriate „C“ program.

(a) lseek() (b) chmod() (c) umask() (d) access() (e) utime()

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/time.h>
#include <stdlib.h>
#include <time.h> // Include time.h for time() function
#include <utime.h> // Include utime.h for utime() and utimbuf struct

int main() {
    // 1. Demonstrating lseek()
    int fd = open("testfile.txt", O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    // Seek to the 10th byte
    off_t offset = lseek(fd, 10, SEEK_SET);
    if (offset == -1) {
        perror("Error seeking in file");
        close(fd);
        return 1;
    }
    printf("lseek() set the file offset to: %ld\n", (long) offset);

    // Write something at the new offset
    write(fd, "Hello, World!", 13);

    // 2. Demonstrating chmod()
    if (chmod("testfile.txt", S_IRUSR | S_IWUSR | S_IRGRP) == -1) {
        perror("Error changing file permissions");
        close(fd);
    }
}
```

```
        return 1;
    }
    printf("chmod() has changed file permissions.\n");

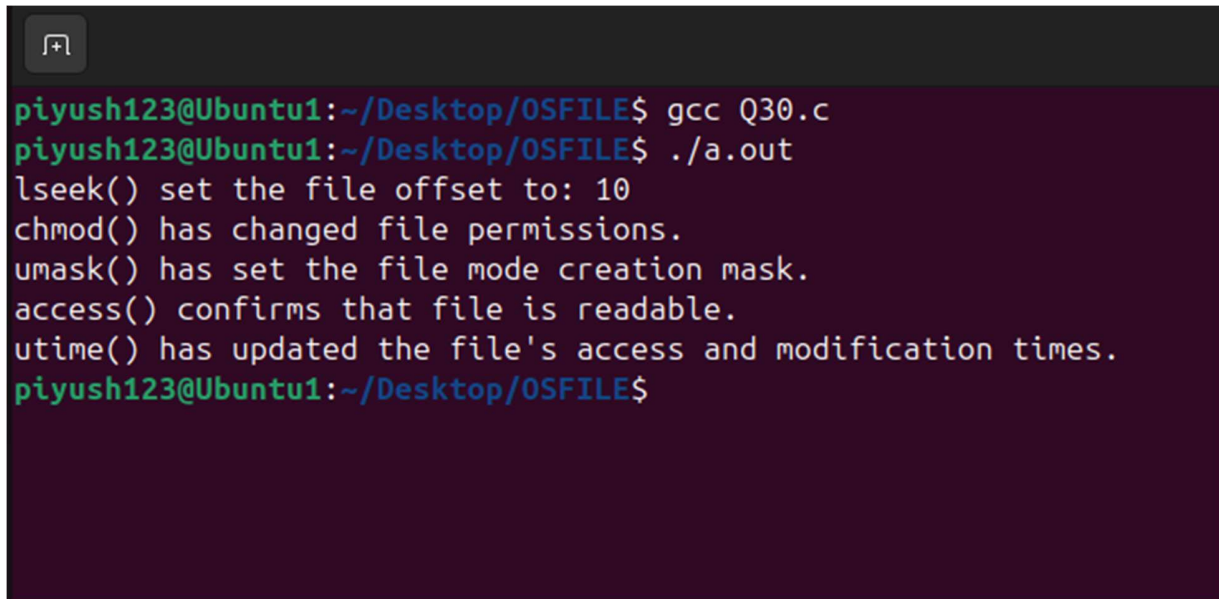
    // 3. Demonstrating umask()
    umask(S_IWGRP); // This will restrict group write permissions by default
    printf("umask() has set the file mode creation mask.\n");

    // 4. Demonstrating access()
    if (access("testfile.txt", R_OK) == 0) {
        printf("access() confirms that file is readable.\n");
    } else {
        perror("access() failed");
    }

    // 5. Demonstrating utime()
    struct utimbuf new_times;
    new_times.actime = time(NULL); // Current time as access time
    new_times.modtime = time(NULL); // Current time as modification time

    if (utime("testfile.txt", &new_times) == -1) {
        perror("Error setting file times");
        close(fd);
        return 1;
    }
    printf("utime() has updated the file's access and modification times.\n");

    // Closing the file descriptor
    close(fd);
    return 0;
}
```


A terminal window with a dark background and light-colored text. The prompt is 'piyush123@Ubuntu1:~/Desktop/OSFILE\$'. The user enters 'gcc Q30.c', followed by './a.out'. The program outputs several lines: 'lseek() set the file offset to: 10', 'chmod() has changed file permissions.', 'umask() has set the file mode creation mask.', 'access() confirms that file is readable.', and 'utime() has updated the file's access and modification times.' The prompt returns at the end.

```
piyush123@Ubuntu1:~/Desktop/OSFILE$ gcc Q30.c
piyush123@Ubuntu1:~/Desktop/OSFILE$ ./a.out
lseek() set the file offset to: 10
chmod() has changed file permissions.
umask() has set the file mode creation mask.
access() confirms that file is readable.
utime() has updated the file's access and modification times.
piyush123@Ubuntu1:~/Desktop/OSFILE$
```

Explanation of Each System Call

1. **lseek()**:
 - Moves the file pointer to a specific location.
 - In this program, we use `lseek(fd, 0, SEEK_SET)` to move the pointer to the beginning of the file before reading.
2. **chmod()**:
 - Changes the permissions of a file.
 - Here, we change the permissions of `example.txt` to 0755 (read, write, execute for owner; read and execute for others).
3. **umask()**:
 - Sets the file mode creation mask, which affects the default permissions of newly created files.
 - We change the `umask` to 0022, which results in new files having at most 0644 permissions.
4. **access()**:
 - Checks the accessibility of a file (e.g., whether it exists or if it's readable/writable).
 - We use `access()` to check if `example.txt` exists and if it has read and write permissions.
5. **utime()**:
 - Updates the access and modification times of a file.
 - Here, we set the access time to one day ago and the modification time to the current time for `example.txt`.